

JSON (JavaScript Object Notation):

JSON is a syntax for storing and exchanging data. JSON is text, written with JavaScript object notation. Python has a built-in package called JSON, which can be used to work with JSON data. JavaScript Object Notation is a lightweight text-based open standard designed for human-readable data interchange. JavaScript Object Notation (JSON) is a human readable and very popular format used by web services, programming languages (including Python) and APIs to read/write data. Import the json module: **Import JSON**. When exchanging data between a browser and a server, the data can only be text. JSON is text, and we can convert any JavaScript object into JSON, and send JSON to the server. JSON is a lightweight data-interchange format. Both JSON and XML can be used to receive data from a web server. JSON is a human readable data format used by applications for storing, transferring and reading data. JSON is a very popular format used by web services and APIs to provide public data. This is because it is easy to parse and can be used with most modern programming languages, including Python. Both JSON and XML can be used to receive data from a web server. XML is much more difficult to parse than JSON. JSON is parsed into a ready-to-use JavaScript object. JSON is light-weight as compare to XML. It is language independent. Easy to read and write. Text based, human readable data exchange format. When we query a device through a script or software application, we can receive that data in JSON format. The data being transferred can be used for machine-to-machine interaction. It's human-readable as well. While we may be used to CLI output, as network engineers, it may not be very useful for machines. Therefore, we have a data structure being used such as JSON. Using the Meraki sandbox as an example, we make a request and we receive a response in JSON format.

JSON Syntax Structure:

- o Uses curly braces {} to hold objects and square brackets [] to hold arrays.
- o JSON data is written as key/value pairs.
- o A key/value pair consists of a key (must be a string in double quotation marks ""), followed by a colon :, followed by a value. For example: "name": "John"
- o Each key must be unique.
- o Values must be of type string, number, object, array, Boolean or null.
- o Multiple key/value within an object are separated by commas ,

JSON Data:

In JSON, the data known as an object is one or more key/value pairs enclosed in braces { }. JSON data is written as name/value pairs, just like JavaScript object properties. A name/value pair consists of a field name (in double quotes), followed by a colon, followed by a value: JSON names require double quotes. JavaScript names do not.

```
"firstName":"Ali"
```

JSON Objects:

JSON objects are written inside curly braces. Just like in JavaScript, objects can contain multiple name/value pairs: Always surrounded by curly brackets. Composed of one-or-more name-value

```
{"firstName":"Ali", "lastName":"Khan"}
```

JSON Arrays:

JSON can use arrays. Arrays are used to store multiple values in a single variable. JSON arrays are written inside square brackets. Just like in JavaScript, an array can contain objects: the object "employees" is an array. It contains three objects. Each object is a record of a person (with a first name and a last name).

```
"employees":[
  {"firstName":"John", "lastName":"Doe"},
  {"firstName":"Anna", "lastName":"Smith"},
  {"firstName":"Peter", "lastName":"Jones"}
]
```

JSON List of IPv4 Addresses:

```
{
  "addresses": [
    {
      "ip": "172.16.0.2",
      "netmask": "255.255.255.0"
    },
    {
      "ip": "172.16.0.3",
      "netmask": "255.255.255.0"
    },
    {
      "ip": "172.16.0.4",
      "netmask": "255.255.255.0"
    }
  ]
}
```

Main JSON Object [

JSON Object ——— {

"capabilities": { **JSON Object inside object**

JSON Array

```
"node_types": [  
  "cloud",  
  "ethernet_hub",  
  "ethernet_switch",  
  "nat",  
  "vpcs",  
  "virtualbox",  
  "dynamips",  
  "frame_relay_switch",  
  "atm_switch",  
  "qemu",  
  "vmware",  
  "traceng"  
],
```

End Object inside object },

```
"compute_id": "local",  
"connected": true,  
"cpu_usage_percent": 29.3,  
"host": "127.0.0.1",  
"last_error": null,  
"memory_usage_percent": 63.7,  
"name": "DESKTOP-3KNIST6",  
"port": 3080,  
"protocol": "http",  
"user": "admin"
```

End Object ——— }

Main JSON Object]

Lab Time:

S1 Configuration

S1#terminal length 0

S1#show running-config | format

When run this command: **show running-config | format** it will show the running configuration in XML format copy the data go to any XML to JSON convertor in google such as <https://codebeautify.org/xmltojson>



Use the DevNet Nexus always on Sandbox:

DNS name: **sbx-nxos-mgmt.cisco.com**

Protocol: **SSH**

Port: **8181**

Username: **admin**

Password: **Admin_1234!**

Example:

Windows: **Use Putty to connect**



